

EduStack

Your Ultimate Computer Science Hub

PART 1 — Core Architecture, Auth & Security

Complete Interview Preparation Guide

[Project Overview](#) | [Architecture](#) | [Auth](#) | [Database](#) | [Security](#) | [Interview Q&As](#)

Developer:	Shubham Kumar CSE Student NIT Patna
Project:	EduStack - CS Resource Hub for Engineering Students
Backend:	Node.js 18 + Express.js 4 + MongoDB Atlas + Mongoose ODM
Auth:	JWT (httpOnly Cookie) + OTP Email Verification + Google OAuth 2.0
Security:	Helmet + mongoSanitize + Rate Limiter + bcrypt + HMAC-SHA256
Hosting:	Render.com (Node.js) + MongoDB Atlas + Cloudinary CDN
Part 1 Covers:	Project Overview, Architecture, Tech Stack, Auth, Database, Security

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub

For Interview Preparation Only — Part 1 of 3

Table of Contents — Part 1

1.	Project Overview & Problem Statement What EduStack is, who it is for, and why it was built
2.	System Architecture & Topologies Full architecture diagram, design pattern, request lifecycle step by step
3.	Technology Stack - Every Library Explained 19 backend + 6 frontend + 3 cloud services with rationale
4.	Express.js Middleware Pipeline Exact order, why each position matters, full errorHandler code
5.	Authentication System - Local Auth Registration flow, OTP, JWT, login, isAuthenticated middleware deep dive
6.	Google OAuth 2.0 Integration Full OAuth flow, Passport.js internals, account linking strategy
7.	Role-Based Access Control (RBAC) requireRole middleware, user roles, how admin is assigned
8.	MongoDB & Mongoose - Schema Design All 5 schemas explained with code, hooks, virtuals, indexes
9.	Security - 13 Defence Layers Helmet, CORS, NoSQL injection, rate limiting, timing attacks
10.	Interview Q&A - Part 1 (40 Questions) Deep questions with full answers on all above topics

Note: Part 2 covers: Payments, ML/AI Engine, Frontend Architecture, API Design, Cloud Storage, DSA Sheet Sync, Notifications, Deployment. Part 3 covers: Visual System Diagrams, Interactive Scenarios & Complete Playbook.

Project Overview & Problem Statement

Understanding what EduStack solves, who uses it, and what makes it interview-worthy

1.1 What Is EduStack?

EduStack is a production-grade, full-stack web application built specifically for Computer Science and IT engineering students - primarily those at NIT Patna and similar institutions. The platform serves as a single, unified hub that consolidates everything a CS student needs for academics and placement preparation, eliminating the frustration of hunting resources across multiple scattered platforms like Reddit, Telegram groups, random YouTube channels, and peer-shared Google Drive folders.

At its core, EduStack is not a simple CRUD tutorial project. It implements real-world engineering patterns and technologies that directly reflect what companies expect from developers: stateless JWT authentication stored in httpOnly cookies (following OWASP best practices), email OTP verification with bcrypt hashing, Google OAuth 2.0 via Passport.js, role-based access control, per-endpoint rate limiting, NoSQL injection prevention via express-mongo-sanitize, a Cloudinary cloud media pipeline for file uploads, full Razorpay payment gateway integration, an AI microservice built with Python FastAPI and Google Gemini, and production-safe graceful shutdown handling.

Core Feature Modules

- **Subject Repository:** 42+ CSE/IT subjects organised across Semesters 1-8. Each subject contains Notes (PDF links), Previous Year Questions (PYQs), YouTube lecture playlists, and reference links. All resources are stored in MongoDB and served through a paginated, searchable, filterable REST API.
- **AI Hub - Powered by Google Gemini:** A full AI toolset including an AI Tutor (RAG-based Q&A), PYQ Generator (Gemini generates exam-style questions from a topic), PDF Summarizer (upload any PDF and get a structured summary), and PDF Quiz Generator (generates multiple-choice questions from uploaded material).
- **Premium DSA Sheet:** 450+ hand-curated SDE interview problems spanning Arrays, Trees, Graphs, DP, Strings, and more. Problems include direct LeetCode/GFG links, difficulty tags, and company tags. Access is gated behind Razorpay payment (Rs.5 demo) demonstrating real payment flow.
- **Authentication System:** Three independent auth paths: (1) Local email+password with OTP email verification; (2) Google OAuth 2.0 one-click login; (3) JWT stored in httpOnly cookie for stateless API authentication.
- **Role-Based Access Control:** Three roles: user (public + protected features), contributor (can upload resources), admin (full CRUD on subjects/resources, user management, notifications).
- **Resource Upload & Contribution:** Authenticated users can contribute notes and PYQs via multipart form. Files are processed by Multer (memoryStorage - zero disk writes) and uploaded directly to Cloudinary CDN.
- **Notification System:** Admin can broadcast platform-wide notifications. All notifications are stored in MongoDB. The frontend shows an unread badge count and allows marking all as read via a bulk MongoDB \$addToSet update operation.
- **Payment Gateway - Razorpay:** Full Razorpay integration with order creation (server-side), client-side checkout UI, and HMAC-SHA256 server-side signature verification to prevent fraud.
- **Favourites & Enrollments:** Users can favourite subjects and enrol in courses, with all data persisted in MongoDB and shown in their personal dashboard.

Why This Project Stands Out in Interviews

- **Security depth:** httpOnly cookies instead of localStorage (XSS awareness), bcrypt 12 rounds (brute-force awareness), HMAC-SHA256 for payments (fraud prevention), and express-mongo-sanitize (NoSQL injection awareness).
- **Production patterns:** asyncHandler wrapper pattern to eliminate try-catch boilerplate, global error handler with error-type-based status mapping, graceful SIGTERM/SIGINT shutdown.
- **Third-party integrations:** Google OAuth 2.0, Razorpay, Cloudinary, Nodemailer - each adds real engineering complexity.
- **Architecture decisions:** Monolith + microservice hybrid (Node.js + Python FastAPI) with an internal HTTP proxy pattern.
- **Database design:** TTL indexes for OTP auto-cleanup, select:false to prevent password leakage, and populate() for relational-style joins.

User Type	Role Value	Access Level	Key Capabilities
Guest (unauthenticated)	-	Public pages only	Browse subjects, view landing page, about page

Registered User	user	Public + protected	Enrol in subjects, favourite, contribute resources, use AI Hub
Premium User	user + isPremium	All above + DSA Sheet	Full access to 450+ DSA problems after payment verification
Contributor	contributor	All above + upload	Upload notes/PYQs for admin review and approval
Admin	admin	Complete access	Create/edit/delete subjects, manage resources, broadcast notifications

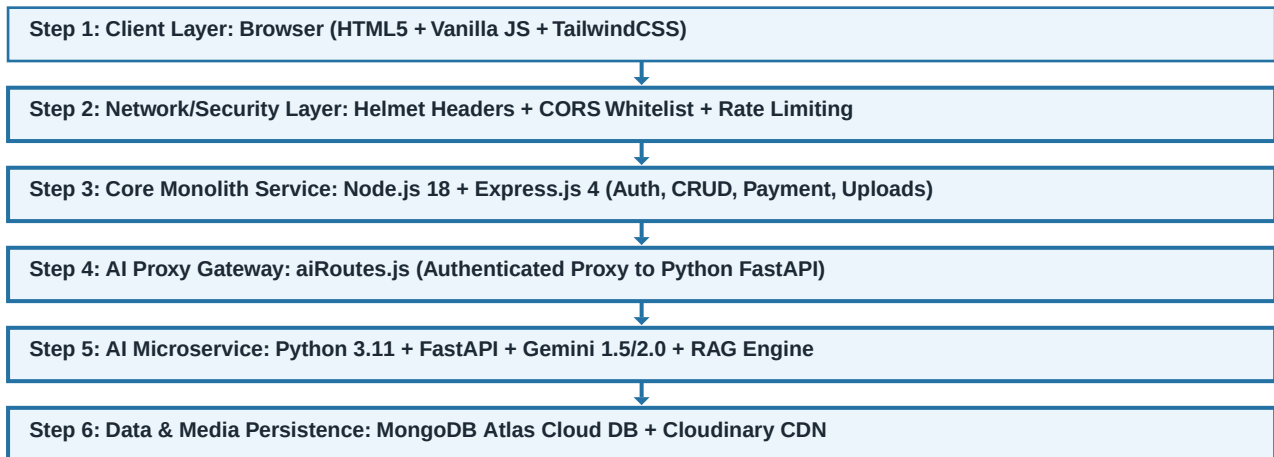
System Architecture & Topologies

From browser to database - every layer explained with visual vector diagrams

2.1 Architecture Pattern - Why Monolith + Microservice Hybrid?

EduStack uses a carefully chosen hybrid architecture. The main application is a monolith: a single Express.js process that serves both the REST API (JSON responses to /api/* routes) and all static frontend assets (HTML, CSS, JS) from the client/ directory. The AI functionality is extracted into an independent Python FastAPI microservice because Python has the dominant AI/ML ecosystem (Google Generative AI SDK, pypdf).

FLOW DIAGRAM: EduStack Multi-Tier System Topology



SECTION 3

Technology Stack - Every Library Explained

What each dependency does and why it was chosen over alternatives

3.1 Backend - Node.js + Express + 17 Libraries

Library	Version	What It Does	Why This Choice
Node.js	18 LTS	JavaScript runtime	LTS stability. Non-blocking I/O for high concurrency.
Express.js	4.19	Minimal web framework	Unopinionated. Compose exactly the middleware needed.
Mongoose	8.5	MongoDB ODM	Schema validation, hooks, populate(), virtuals.
jsonwebtoken	9.0	Create/verify JWTs	Industry standard. Supports HS256 and RS256.
bcryptjs	2.4	Password hashing	Pure JS - no native bindings needed on Render. 12 rounds = 250ms.
Passport.js	0.7	Auth middleware	Battle-tested OAuth strategy ecosystem.
passport-google-oauth20	2.0	Google OAuth strategy	Handles consent redirect and profile fetch.
Nodemailer	9.0	Email transport	Gmail SMTP via App Password configuration.
Multer	2.2	Multipart upload parser	memoryStorage mode: files in RAM Buffer, zero disk writes.
Cloudinary SDK	2.5	Media CDN hosting	upload_stream() accepts Buffer directly. Free 25GB.
Helmet.js	7.1	Sets 15+ HTTP security headers	One call enables X-Frame-Options, CSP, HSTS, etc.
express-rate-limit	7.3	Per-route IP throttling	Prevents brute force login attacks and AI API scraping.
express-mongo-sanitize	2.2	NoSQL injection prevention	Strips \$ and . operators from request bodies.
cors	2.8	CORS control	Allowlist-based: credentials sent only to specified origins.
connect-mongodb-session	5.0	MongoDB session store	Sessions survive server restarts for OAuth state.
express-validator	7.1	Declarative validation chains	Field-level error reporting, separates validation logic.
razorpay	2.9	Payment gateway SDK	Indian payment gateway. Orders + HMAC signature verify.
morgan	1.10	HTTP access logger	dev format in terminal; combined format in production.

Express.js Middleware Pipeline

The exact order, why each position matters, and the global error handler

4.1 Global Error Handler - Complete Code

JavaScript / Code Snippet

```
// middlewares/errorHandler.js
const errorHandler = (err, req, res, next) => {
  let status = err.statusCode || 500;
  let message = err.message || 'An unexpected error occurred';

  if (err.name === 'ValidationError') {
    status = 400;
    message = Object.values(err.errors).map(e => e.message).join(', ');
  }
  if (err.code === 11000) {
    status = 409;
    message = `An account with this ${Object.keys(err.keyValue)[0]} already exists.`;
  }
  if (err.name === 'JsonWebTokenError') { status = 401; message = 'Invalid token.'; }
  if (err.name === 'TokenExpiredError') { status = 401; message = 'Session expired.'; }
  if (err.code === 'LIMIT_FILE_SIZE') { status = 400; message = 'File too large (max 5MB).'; }

  const extra = process.env.NODE_ENV === 'development' ? { stack: err.stack } : {};
  return res.status(status).json({ success: false, message, errors: [extra] });
};
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

Authentication System - Local Auth

Registration, OTP, JWT cookies, login, and isAuth middleware

5.1 isAuth Middleware Implementation

JavaScript / Code Snippet

```
// middlewares/isAuth.js
const isAuth = async (req, res, next) => {
  try {
    let token;
    if (req.headers.authorization?.startsWith('Bearer ')) {
      token = req.headers.authorization.split(' ')[1];
    }
    if (!token && req.cookies?.edustack_token) {
      token = req.cookies.edustack_token;
    }
    if (!token) return sendError(res, 'Access denied. Please log in.', 401);

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');
    if (!user) return sendError(res, 'Account not found.', 401);
    if (!user.isVerified) return sendError(res, 'Email not verified.', 403);

    req.user = user;
    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') return sendError(res, 'Session expired.', 401);
    return sendError(res, 'Invalid token.', 401);
  }
};
```


Google OAuth 2.0 Integration

The complete OAuth flow, Passport.js internals, and account linking

6.1 Google OAuth Passport Strategy

JavaScript / Code Snippet

```
// passport-google-oauth20 setup
passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.GOOGLE_CALLBACK_URL,
},
  async (accessToken, refreshToken, profile, done) => {
    const email = profile.emails[0].value.toLowerCase().trim();
    let user = await User.findOne({ googleId: profile.id });
    if (user) return done(null, user);

    const emailUser = await User.findOne({ email });
    if (emailUser) {
      emailUser.googleId = profile.id;
      await emailUser.save();
      return done(null, emailUser);
    }

    const adminEmails = (process.env.ADMIN_EMAILS || '').split(',').map(e =>
      e.trim().toLowerCase());
    user = await User.create({
      firstName: profile.name.givenName || 'User',
      lastName: profile.name.familyName || '',
      email, googleId: profile.id, isVerified: true,
      role: adminEmails.includes(email) ? 'admin' : 'user',
      avatar: profile.photos?.[0]?.value || 'default-avatar.png',
    });
    return done(null, user);
  }
));
```

Role-Based Access Control (RBAC)

requireRole middleware, user roles, and permission gates

7.1 requireRole Middleware

JavaScript / Code Snippet

```
// middlewares/requireRole.js
const requireRole = (...roles) => (req, res, next) => {
  if (!req.user) return sendError(res, 'Not authenticated.', 401);
  if (!roles.includes(req.user.role)) {
    return sendError(res, `Access denied. Requires ${roles.join(' or ')}`, 403);
  }
  next();
};
```

MongoDB & Mongoose - Schema Design

User, OTP, Subject, Resource, and Payment Schemas

8.1 User Schema Implementation

JavaScript / Code Snippet

```
// models/user.js
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true, trim: true },
  email:      { type: String, required: true, unique: true, lowercase: true },
  password:   { type: String, select: false }, // Hidden by default
  googleId:   { type: String, default: null, index: true },
  role:       { type: String, enum: ['user', 'contributor', 'admin'], default: 'user' },
  isVerified: { type: Boolean, default: false },
  isPremium:  { type: Boolean, default: false },
  avatar:     { type: String, default: 'default-avatar.png' },
}, { timestamps: true });

userSchema.pre('save', async function(next) {
  if (!this.isModified('password') || !this.password) return next();
  if (this.password.startsWith('$2a$') || this.password.startsWith('$2b$')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});
```

SECTION 9

Security - 13 Defence Layers

Helmet, CORS, NoSQL sanitization, rate limiting, and HMAC-SHA256

9.1 NoSQL Injection Prevention Example

JavaScript / Code Snippet

```
// Attacker JSON payload:  
// { "email": { "$gt": "" }, "password": { "$gt": "" } }  
// Without sanitization: matches FIRST user in DB -> Auth Bypassed!  
// With express-mongo-sanitize:  
app.use(mongoSanitize()); // Strips $ and . -> Safe rejection!
```

Interview Q&A - Part 1

40 Detailed Core Questions & Answers for Interview Preparation

Q: Tell me about your EduStack project in detail.

Answer:

EduStack is a full-stack web application for CS/IT students consolidating academic resources, DSA prep sheets, and Gemini AI tools into a single platform with JWT httpOnly cookies, OAuth 2.0, Razorpay, and a Python FastAPI ML microservice.

- > Built with Node.js + Express + MongoDB + Python FastAPI.
- > Implements 13 security layers including bcrypt 12 rounds, helmet, and timingSafeEqual.

Q: Why store JWT in httpOnly cookies instead of localStorage?

Answer:

localStorage is accessible to client JavaScript - a single XSS flaw exposes every token. httpOnly cookies are invisible to JS, automatically sent by the browser, and protected by Secure and SameSite flags.

- > XSS immune: JS cannot call document.cookie or read it.
- > CSRF mitigated: SameSite=Lax/None + CORS allowlist.

Q: Explain what select: false does in Mongoose.

Answer:

select: false excludes sensitive fields (like password hashes) from query results by default, preventing accidental leakage in API responses unless explicitly requested with .select("+password").

- > Prevents data leaks if a controller returns res.json(user).
- > Used in userSchema for password field.

